

Representación de posición y orientación

Instalacion Toolbox Robotics de Peter Corke

1. Para instalar el toolbox robotics de Peter Corke puede descargarlo desde la pagina: <https://petercorke.com/toolboxes/robotics-toolbox/>
2. Copie el archivo RTB.mltbx en el espacio de trabajo, mediante doble-click en el mismo ejecute la instalacion del toolbox.
3. Finalmente usando la aplicacion "pathtool" desde el command Window seleccione los path realcionados al "Robotics Toolbox for MATLAB" y subalos al tope de la lista. El no realizar esta ultima accion puede a que funciones de la libreria sean reemplazadas por versiones de matlab que no se comportand de la misma manera.

Representacion en 2D

Matriz de rotacion 2D:

Representacion de matriz de rotacion en 2D:

```
R=rot2(0.7) %radianes
```

```
R = 2x2
    0.7648    -0.6442
    0.6442     0.7648
```

O en forma simbolica:

```
syms theta
R=rot2(theta)
```

```
R =
( cos(theta)  -sin(theta) )
( sin(theta)   cos(theta) )
```

Transformacion homogenea 2D:

transl gerenara una translacion pura

trot genera una rotacion pura

```
T1=transl2(1,2)*trot2(30,'deg') %transalacion 1x 2y y giro 30 grados
```

```
T1 = 3x3
    0.8660    -0.5000     1.0000
    0.5000     0.8660     2.0000
         0         0     1.0000
```

Podemos crear un grafico para ver estos giros utilizando:

```
plotvol([0 5 0 5]);
trplot2(T1, 'frame', '1', 'color','b')
```

Observemos que pasa si creamos otra transformacion homogenea y probamos su multiplicacion (composicion):

```
T2=transl2(2,1)
```

```
T2 = 3x3
```

```
  1  0  2
  0  1  1
  0  0  1
```

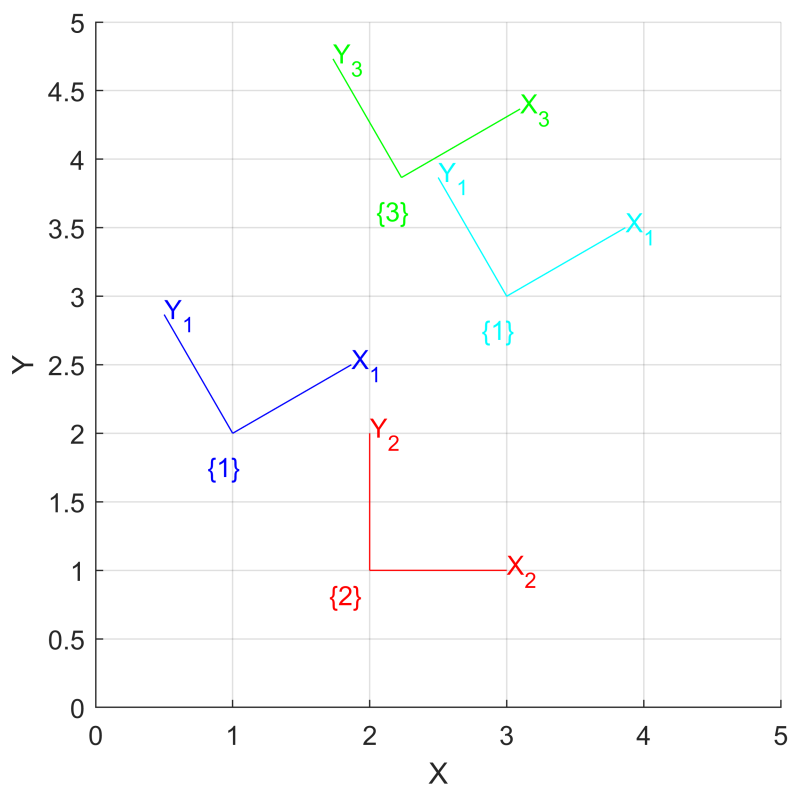
```
trplot2(T2, 'frame', '2', 'color','r')
```

```
T3=T1*T2;
```

```
trplot2(T3, 'frame', '3', 'color','g')
```

```
T4=T2*T1;
```

```
trplot2(T4, 'frame', '1', 'color','c')
```



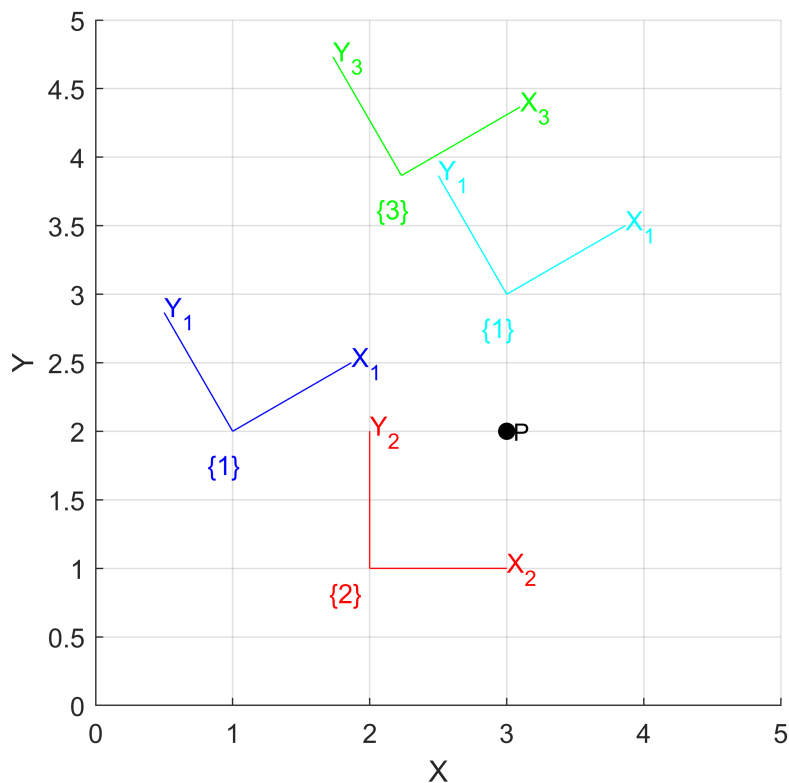
Vemos que **la composicion es no conmutativa**

Punto

Podemos definir un punto como :

```
P=[3;2];
```

```
plot_point(P, 'label', 'P', 'solid', 'ko')
```



Supongamos que el punto anterior esta definido en el marco 0 que venimos trabajando, si deseamos convertirlo ala marco 1 por ejemplo deberiamos convertirlo a coordenadas homogeneas y aplicar la tranformacion correspondiente:

$${}^1P = {}^1\xi_0 \cdot {}^0P$$

```
P1o=inv(T1)*[P;1]
```

```
P1o = 3x1
    1.7321
   -1.0000
    1.0000
```

Una forma equivalente es usar las funciones e2h y h2e para convertir y deconvertir de coordenadas euclideas a homogeneas:

```
P1=h2e(inv(T1)*e2h(P))
```

```
P1 = 2x1
    1.7321
   -1.0000
```

Centros de rotación

Hasta ahora conocemos como hacer una rotacion con respecto al marco origen, pero ¿Como resolvemos esto si queremos realizarlo respecto de un punto C generico?

Para esto primero deberemos llevar lo que nos interese rotar al marco origen, luego rotarlo y luego volverlo al marco de interes. Vemos el siguiente ejemplo:

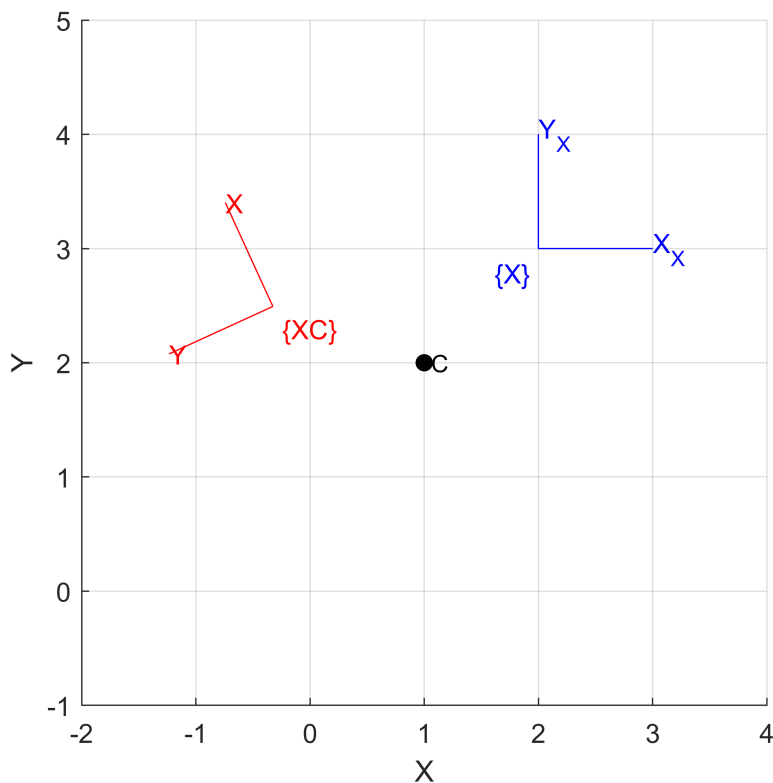
```
plotvol([-2 4 -1 5])
C=[1 2]'; %punto de rotacion
plot_point(C, 'label', 'C', 'solid', 'ko')
%definimos un marco desplazado
X=transl2(2,3);
trplot2(X, 'frame', 'X');
%definimos transformacion de rotacion
R=trot2(2)
```

```
R = 3x3
    -0.4161    -0.9093         0
     0.9093    -0.4161         0
         0         0         1.0000
```

```
%realizamos transformacion
RC=transl2(C)*R*transl2(-C)
```

```
RC = 3x3
    -0.4161    -0.9093    3.2347
     0.9093    -0.4161    1.9230
         0         0         1.0000
```

```
trplot2(RC*X, 'framelabel', 'XC', 'color','r')
```



Nota util: para leer la transformación RC podemos hacerlo de derecha a izquierda.

Otra alternativa utilizar las Twists: (veremos mas adelante)

Representacion en 3D

Representacion rotacion 3D. Matrices de rotación ortonormal:

Podemos obtener las matrices de rotacion ortonormal respecto a cada eje con las funciones (estas funciones no soportan representacion simbolica):

```
syms theta
Rx=rotx(theta) %rotacion respecto eje x
```

$$R_x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{pmatrix}$$

```
Ry=roty(theta) %rotacion respecto eje y
```

$$R_y = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{pmatrix}$$

```
Rz=rotz(theta) %rotacion respecto eje z
```

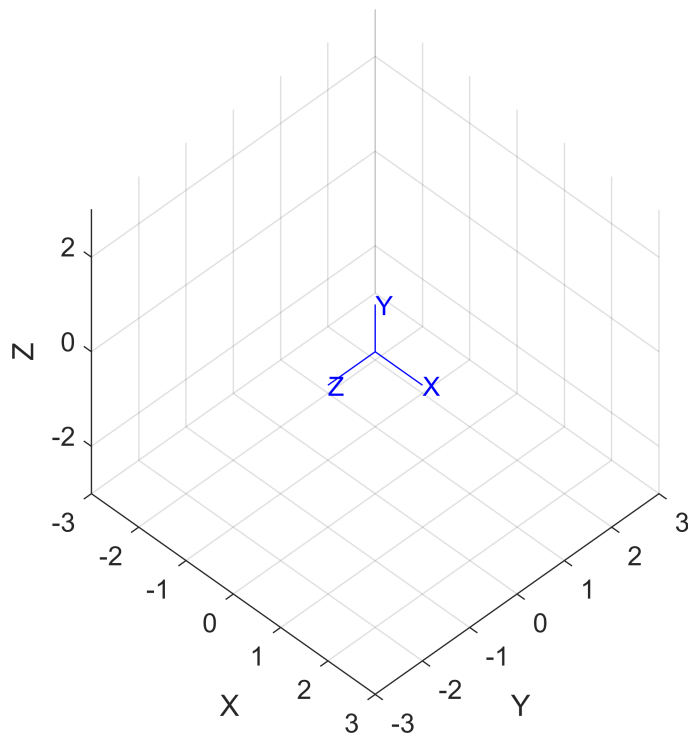
$$R_z = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Podemos aprovechar para ver una animacion de la rotacion mediante los comandos:

```
R=rotx(pi/2) %rotacion de 90 grados respecto eje x
```

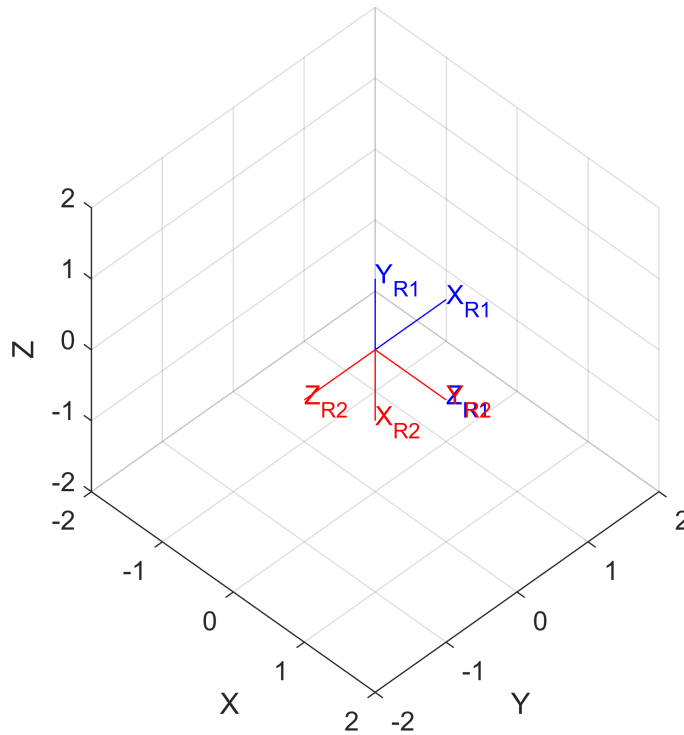
```
R = 3x3
     1     0     0
     0     0    -1
     0     1     0
```

```
plotvol(3)
%trplot(R) %para ver posicion final
view([45 45])
tranimate(R) %para ver animación - ejecutar en Command Window
```



Podemos aprovechar a ver como la **composicion de rotaciones no es conmutativa**:

```
plotvol(2)
view([45 45])
R1=rotx(pi/2)*roty(pi/2);
R2=roty(pi/2)*rotx(pi/2);
trplot(R1, 'frame', 'R1', 'color','b')
trplot(R2, 'frame', 'R2', 'color','r')
```



Representacion rotacion 3D. Representacion de 3 angulos - Angulos de Euler

Secuencia de ángulos ZYZ: comúnmente utilizada en aeronáutica y análisis dinámicos mecánicos (utilizada en el toolbox).

$$R = R_z(\phi)R_y(\theta)R_z(\psi)$$

El marco resultante se obtiene de la composición de rotaciones con respecto de los **marcos actuales**, y puede calcularse como la **post-multiplicación** de matrices elementales de rotación:

```
%calculo de la matriz de rotacion
```

```
R=rotz(0.1)*roty(0.2)*rotz(0.3)
```

```
R = 3x3
```

```
    0.9021    -0.3836     0.1977
    0.3875     0.9216     0.0198
   -0.1898     0.0587     0.9801
```

```
%o en forma equivalente
```

```
R=eul2r(0.1,0.2,0.3)
```

```
R = 3x3
```

```
    0.9021    -0.3836     0.1977
    0.3875     0.9216     0.0198
   -0.1898     0.0587     0.9801
```

```
%podemos encontrar los angulos de euler a partir de la matriz de rotacion
```

```
gamma=tr2eul(R)
```

```
gamma = 1x3  
    0.1000    0.2000    0.3000
```

Analizar que pasa si la rotacion en el eje Y es negativa:

```
R=eul2r(0.1,-0.2,0.3)
```

```
R = 3x3  
    0.9021   -0.3836   -0.1977  
    0.3875    0.9216   -0.0198  
    0.1898   -0.0587    0.9801
```

```
%podemos encontrar los angulos de euler a partir de la matriz de rotacion  
gamma=tr2eul(R)
```

```
gamma = 1x3  
   -3.0416    0.2000   -2.8416
```

Podemos ver que para este caso el resultado parece no corresponder a los angulo originales, el problema aqui es que **el mapeo desde la matriz de rotacion a los angulos de Euler no es unico** y el toolbox siempre devuelve el angulo de rotacion respecto de Y positivo.

¿Que pasa si $\theta = 0$?

```
delta=0.01 %probar distintos valores y ver que pasa con la inversa 0.01,-0.01,0, etc
```

```
delta = 0.0100
```

```
R=eul2r(0.1,0+delta,0.3)
```

```
R = 3x3  
    0.9210   -0.3894    0.0099  
    0.3894    0.9211    0.0010  
   -0.0096    0.0030    1.0000
```

```
%podemos encontrar los angulos de euler a partir de la matriz de rotacion  
gamma=tr2eul(R)
```

```
gamma = 1x3  
    0.1000    0.0100    0.3000
```

Vemos que en este caso $R_y(\theta) = I$ entonces $R = R_z(\phi)R_z(\psi) = R_z(\phi + \psi)$, la inversa no puede hacer el calculo y toma por defecto $\phi=0$. **Estamos en una singularidad.**

Secuencia de ángulos ZYX (Roll-Pitch-Yaw): comúnmente utilizada en aeronáutica y manipuladores.

$$R = R_z(\theta_y)R_y(\theta_p)R_x(\theta_r)$$

El marco resultante se obtiene de la composición de rotaciones con respecto del **marco fijo original**, y puede calcularse como la **pre-multiplicación** de matrices elementales de rotación:

```
%calcula de la matriz de rotacion  
R=rotx(0.1)*roty(0.2)*rotz(0.3)
```



```
R = 3x3
    0.9363   -0.2896    0.1987
    0.3130    0.9447   -0.0978
   -0.1593    0.1538    0.9752
```

```
%o en forma equivalente
R=ropy2r(0.1,0.2,0.3,'zyx')
```

```
R = 3x3
    0.9363   -0.2751    0.2184
    0.2896    0.9564   -0.0370
   -0.1987    0.0978    0.9752
```

```
%podemos encontrar los angulos de euler a partir de la matriz de rotacion
gamma=tr2rpy(R,'zyx')
```

```
gamma = 1x3
    0.1000    0.2000    0.3000
```

¿Que pasa si $\theta_p = \pm \frac{\pi}{2}$?

```
delta=0.001 %probar distintos valores y ver que pasa con la inversa 0.01,-0.01,0, etc
```

```
delta = 1.0000e-03
```

```
R=ropy2r(0.1,pi/2+delta,0.3,'zyx')
```

```
R = 3x3
   -0.0010   -0.1987    0.9801
   -0.0003    0.9801    0.1987
   -1.0000   -0.0001   -0.0010
```

```
gamma=tr2rpy(R,'zyx')
```

```
gamma = 1x3
   -3.0416    1.5698   -2.8416
```

Vemos que en este caso $R_y(\theta_p)$ me genera una alineacion de los planos de rotacion z y x, portanto estoy en una condicion de singularidad. **Estamos en una singularidad.**

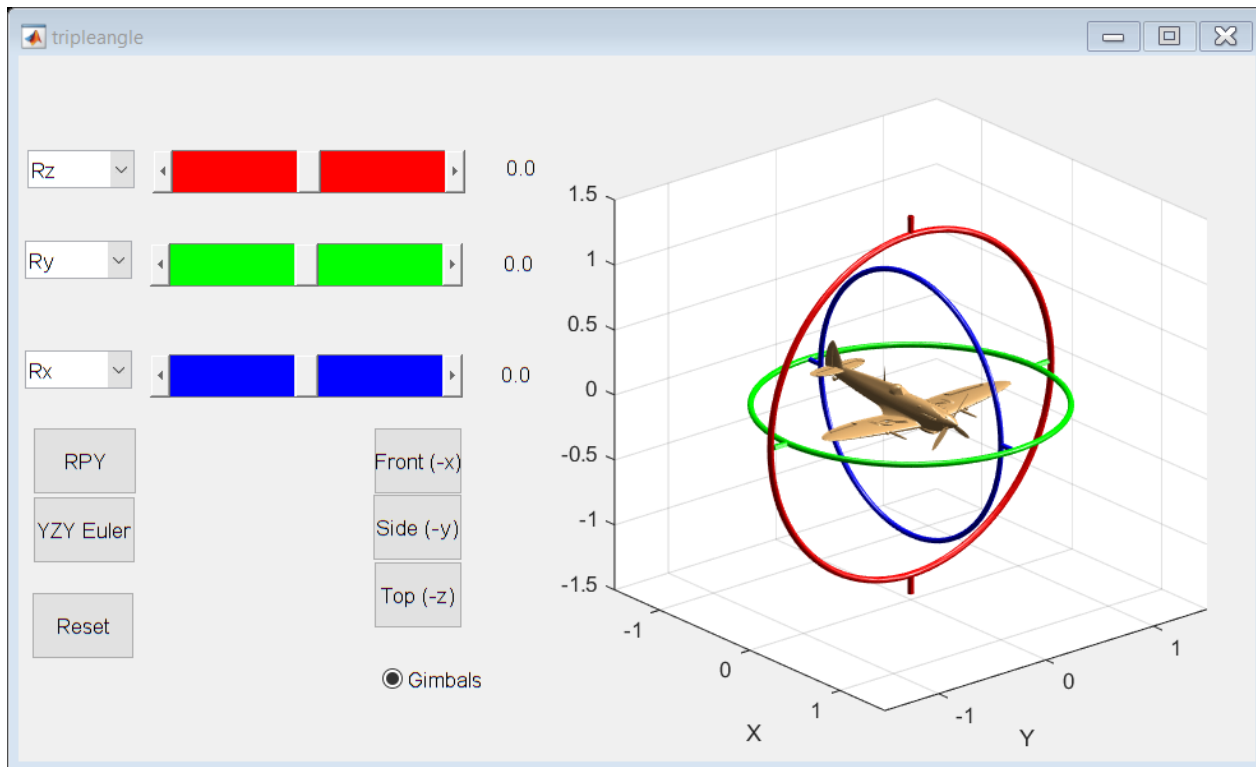
Singularidades de representacion de tres angulos - Gimbal Lock:

Podemos ver de lo anterior que el problema de singularidad en la representacion de 3 angulos siempre ocurre cuando el eje de rotacion del termino central de la secuencia se vuelve paralelo al eje de rotacion del primer o tercer termino (*o dicho de otra forma siempre que 2 ejes de rotacion consecutivos se alinean*).

Para visualizar los casos anteriores puede utilizar la siguiente herramienta del toolbox:

```
tripleangle
```

```
reading STL models.....
Supermarine Spitfire Mk VIII by Ed Morley @GRABCAD
Gimbal models by Peter Corke using OpenSCAD
```



Representacion rotacion 3D. mediante 2 vectores:

En algunas aplicaciones se utiliza un vector de aproximacion a para representar una rotacion, aunque por si solo no lo puede hacer asi que se le asocia un segundo vector llamado de orientacion o , estos dos vectores definen completamente una matriz de rotacion:

$$a = [1 \ 0 \ 0]^T$$

$$a = \begin{matrix} 3 \times 1 \\ 1 \\ 0 \\ 0 \end{matrix}$$

$$o = [0 \ 1 \ 0]^T$$

$$o = \begin{matrix} 3 \times 1 \\ 0 \\ 1 \\ 0 \end{matrix}$$

$$R = \text{oa2r}(o, a)$$

$$R = \begin{matrix} 3 \times 3 \\ \begin{matrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{matrix} \end{matrix}$$

Representacion rotacion 3D usando un vector y un ángulo:

Toda rotacion entre dos marcos puede representarse con un vector y un ángulo de giro en el eje del mismo.

```
R=ropy2r(0.1,0.2,0.3) %una matriz de rotacion cualquiera
```

```
R = 3x3
    0.9363    -0.2751    0.2184
    0.2896    0.9564   -0.0370
   -0.1987    0.0978    0.9752
```

```
% podemos determinar el angulo y vector que representan esta rotacion
% mediante:
[theta,v]=tr2angvec(R)
```

```
theta = 0.3655
v = 1x3
    0.1886    0.5834    0.7900
```

Esta informacion esta contenida en los autovalores λ y autovectores v de la matriz R. Recordano su definicion:

$$Rv = \lambda v$$

Para el caso que $\lambda = 1$, que es el caso para las matrices de rotacion por ser ortonormales, luego v es el vector que permanece sin cambios en la rotacion (o sobre el que se realiza el giro).

Podemos realizar el camino inverso, a partir del angulo y el vector obtener la matriz de rotacion (seria la aplicacion de la **Formula de Rodrigues**):

```
R=angvec2r(pi/2,[1 0 0])
```

```
R = 3x3
    1.0000         0         0
         0    0.0000   -1.0000
         0    1.0000    0.0000
```

Representacion rotacion 3D, usando matriz exponencial:

Podemos comprobar que una matriz de rotacion puede expresarse utilizando la matrix expnonecial de la siguiente forma:

1) Partimos de un ratoacion cualquiera:

```
R=rotx(0.3)
```

```
R = 3x3
    1.0000         0         0
         0    0.9553   -0.2955
         0    0.2955    0.9553
```

2) Calculamos el logaritmo natural de la matriz resultante (recordar que el logaritmo es la inversa de al exponente):

```
S=logm(R)
```

```
S = 3x3
         0         0         0
         0         0   -0.3000
         0    0.3000         0
```

3) Vemos que el resultado es una matriz sesgada simetrica, en particualr podemos obtener el vector de la matriz como:

```
vex(S) '
```

```
ans = 1x3
      0.3000      0      0
```

4) Ver que obtenemos el vector original de la rotacion.

5) Con lo anterior en mente vemos que podemos escribir la forma exponencial siendo equivlente a la primera rotacion de la que partimos:

```
R=expm(skew([1 0 0])*0.3) % la funcion skew crea la matriz sesgada simetrica a partir de un vec
```

```
R = 3x3
      1.0000      0      0
      0      0.9553 -0.2955
      0      0.2955  0.9553
```

```
%la inversa a esta accion pude obtenerse como:
[th,w]=trlog(R)
```

```
th = 0.3000
w = 3x1
      1.0000
      0
      0
```

Representación de rotación 3D: Cuaterniones unitarios

El toolbox ofrece la clase UnitQuaternion para su representacion, la misma admite distintas funciones:

```
% convertir una matriz de rotacion en cuaternion unitario:
q=UnitQuaternion(rpy2tr(0.1,0.2,0.3))
```

```
q =
0.98335 < 0.034271, 0.10602, 0.14357 >
```

```
% multiplicacion de cuaterniones
qm=q*q
```

```
qm =
0.93394 < 0.0674, 0.20851, 0.28236 >
```

```
% inversion
inv(q) %multiplicar a un cuaternion por su inversa lleva al cuaternion unitario q*inv(q)
```

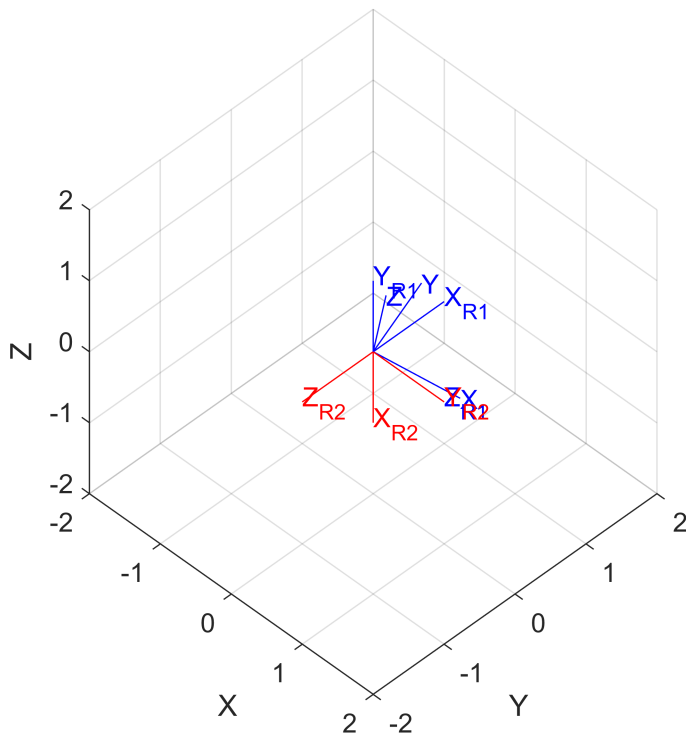
```
ans =
```

```
0.98335 < -0.034271, -0.10602, -0.14357 >
```

```
% convertir un cuaternion en un amatriz de rotacion ortonormal  
q.R
```

```
ans = 3x3  
    0.9363   -0.2751    0.2184  
    0.2896    0.9564   -0.0370  
   -0.1987    0.0978    0.9752
```

```
%representar la orientacion del cuaternion en una figura  
q.plot()
```



```
%por ejemplo si deseamos rotar un vector por un cuaternion simplemente  
%recurrirnos al operador de multiplicacion:  
v=[1 0 0]'
```

```
v = 3x1  
    1  
    0  
    0
```

```
vr=q*v
```

```
vr = 3x1  
    0.9363  
    0.2896  
   -0.1987
```

Representacion pose 3D: Matriz de trasformación homogenea.

El toolbox soporta la representacion de pose mediante matriz de trasformacion homogenes, para esto podemos componer la pose mediante las fuciones transl (crea pose relativa con solo desplazamiento pero no rotacion) y trotx, troty, trotx (crea rotaciones puras sin trasnlacion respecto de cada eje).

Ver la siguiente transformacion obtenida de mover x en una unidad, luego girar 90 grados respecto del eje x y finalmente desplazarse en una unidad en el nuevo eje y.

```
T=transl(1,0,0)*trotx(pi/2)*transl(0,1,0)
```

T = 4x4

1	0	0	1
0	0	-1	0
0	1	0	1
0	0	0	1

```
plotvol(2)  
trplot(T)  
view([45 45])
```

Ejemplo de aplicacion del mundo de la localizacion:

Un robot se encuentra navegando por un mundo 2D. A travez de su odometria (producida por la integracion de la lectura de sus encoders) conocemos que su posicion es 10 metros en el eje x y 5 metros en el eje y, con una orientacion de 45 grados en el sentido del reloj. Esta posicion se la conoce como marco "base link" en el marco de referencia "odom".

El modulo de localizacion, luego de procesar datos de distintos sensores, ha estimado la verdadera posicion del robot en el marco "map" como 12 metros en la direccion del eje x y 4 metros en la direccion del eje y, con una orientacion de 30 grados en el sentido del reloj.

Determine la pose del marco "odom" en el marco "map". Realice una grafico con los tres marcos intervinientes.

```
Tm=transl2(0,0); %marco map  
o_T_bl=transl2(10,5)*trotx(-45,'deg'); %odom a base link  
m_T_bl=transl2(12,4)*trotx(-30,'deg'); %map a base link  
  
m_T_o=m_T_bl*inv(o_T_bl);  
  
plotvol([0 15 -6 6]);  
hold on;  
trplot2(Tm, 'map', '1', 'color','k')  
trplot2(m_T_o*o_T_bl, 'base_link', '1', 'color','b')  
trplot2(m_T_o, 'odom', '1', 'color','r')
```

